

Cat Café / Clowder AI 文章核心摘要（整合版）

Summary: 本文聚焦于：“如何建立 Multi-Agent Harness 工程”这一核心问题。本文认为：Harness 不应该替模型思考，而应该让模型在正确的坐标系里思考。在工程实践的角度，回答了：Multi-Agent 的循环执行 (CH2.)，现实能力 (CH3.)，团队记忆系统构建 (CH4.)，Harness 的评估标准 (CH5.)，可靠性建设 (CH6.) 等复杂问题，并总结了基于能力画像 (Capability Profile) 构建的 Multi-Agent 带来的丰富能力涌现。最终，得出朴素的工程结论：好的 Agent 系统不是把单点能力推到极限，而是把单点波动组织进一个会自我校准的伙伴系统

CH0. Intro - 从 Role Agent 到 Capability Profile

0.1 搭建一个 Multi-agent 系统，会很自然地想到先分配好岗位 (Role Agent)：产品经理 agent、项目经理 agent、开发 agent... 然后绑定不同的模型、工具、技能、提示词，像公司组织架构一样。但是，大语言模型是一个更高维度的智能体，岗位的固定化限制，会压缩大模型的能力；

0.2 Anthropic 定义了五种多 Agent 协作模式：**Generator-Verifier** (生产者-验证者)、**Orchestrator-SubAgent** (编排者-子 Agent)、**Agent Teams** (长期团队)、**Message Bus** (消息总线)、**Shared State** (共享状态) 我们发现了一些问题：

- **模式会叠加。**同一个交互行为，从不同层面看是不同模式。具体的任务可能兼具上述多种的形态；
- **模式会自动切换。**同一个 Agent 在同一个任务里会跨模式。模式的边界不是在配置文件定义的，是交互对象的自主性决定的；
- **缺少统一的指挥者。**五种模式描绘的是 Agent 之间的协作关系，但是整体方向校准发生在 Agent 系统之外；

0.3 我们的做法：先要给 Agent 建立能力画像 (Capability Profile)，再给任务建立任务画像，最终在运行时动态匹配职责。角色概念仍然存在，只是任务的临时职责，不是长期身份。主体画像，来自模型固有能力、认知风格、工具边界、历史表现、坏直觉和当前状态；

0.4 这些维度的可变性并不一样：原生能力、认知风格和坏直觉更接近模型层常量；skill、collection 和工具挂载是变量；历史表现会随任务单调积累；当前状态则是瞬时信号。Role-Agent 最容易抓住的是"给谁加载什么知识"这一层，但最容易漏掉的，恰恰是常量、历史和瞬时状态这三层。

CH1. Agent 能力的定义 – 核心公式：LLM 模型能力 × Harness 契合度

1.1 核心公式：

Agent 质量 = LLM 模型能力 × Harness 契合度 (Environment Fit)

- 这里的 Harness 指的是**围绕模型的整套运行骨架**：持久状态层、验证回路、协作协议、可观测链路、恢复机制。就像赛车的底盘和调校系统，引擎（模型）再强，底盘拉胯就跑不出成绩；
- 同一只 Agent 放进不同 harness，发挥的能力完全不同，本文认为 harness 工程的乘数效应被严重低估了；

1.2 Agent 是由一个它与现实之间的闭环定义的：

观察(现实状态) → 推理(计算状态) → 行动 → 写回(现实状态') → 验证

没有这个闭环，记忆系统只是一个没人查的数据库。没有现实状态，模型只是在隔绝中推理——强大的认知与后果脱节。把两者连起来的那个闭环，才让一个 Agent 是在现实里行动，而不只是有能力。

这里有一个直接的工程含义：**每一分花在让闭环更紧的钱——更快的观察、更好的验证、更丰富的现实状态——都会在每一次模型升级时免费复利**。更好的模型立刻受益于已有环境。但更好的模型放进光秃秃的环境，每个会话都得从零重新推导一切。

层	这里存什么	存续时间	谁控制
权重状态	训练写进的参数	直到下次训练前持久	模型厂商
计算状态	KV cache、隐藏激活	单次推理调用	模型架构

层	这里存什么	存续时间	谁控制
现实状态	代码仓、git 历史、文档、任务归属、记忆	跨推理、跨 Agent、跨时间	环境层 (harness)

表：Agent 会触及的三层状态。前两层是模型厂商的领地，第三层是 harness 工程操作的地方

1.3 Harness 的投资存在两种半衰期，有些随模型变强而折旧，有些随模型变强而增值：

- **Build to Delete（有保质期的脚手架）**：弥补模型当前认知缺陷的代码。这些东西当下 ROI 可能是全场最高的，但它们有**保质期**：当下一代模型原生掌握了这些能力，脚手架就成了噪音。
- **Built to Persist（复利型基础设施）**：编码外部现实、协作协议和可验证边界的代码。这些不是在补模型的短板，是在建模型与现实之间的桥。模型越强，这些桥越值钱。

Build to Delete（有保质期的脚手架）	Built to Persist（复利型基础设施）
详细的思维链模板	文件系统 / git / 搜索工具接入
多步推理引导	trace 基础设施与可观测性
错误恢复样板代码	测试 / lint / review 反馈回路
工具调用别名兜底	Agent 交接协议与路由
人格装饰文字	不可逆操作护栏与应急开关

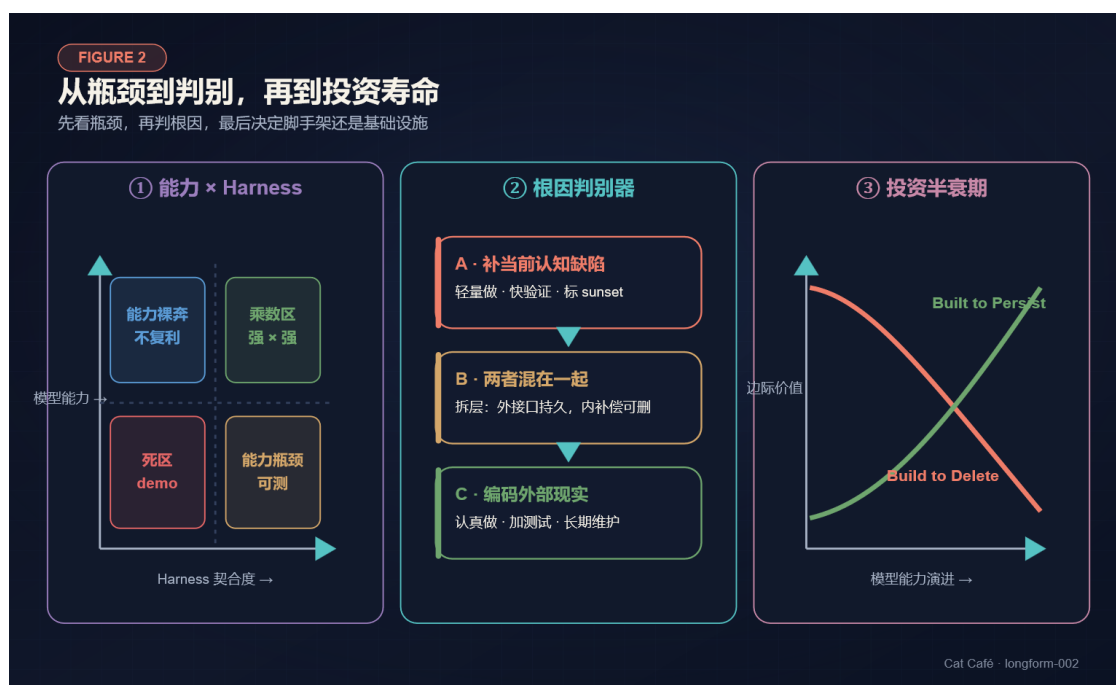
表：Harness 投资的两种半衰期

1.4 两种投资都应该做。但是把脚手架当永久基础设施来精装修——是一种错误。很多人不知道自己写的哪些是脚手架，因而对旧资产过度投资，区分两者是一个判别式：

这层 harness 是在补模型当前的认知缺陷，还是在编码外部现实和协作协议？

- 前者→轻量做、快验证、标好 sunset 时间——它是救火队，随时准备光荣退役。
- 后者→认真做、加测试、写文档——它是地基，每一代模型都站在上面。

- 分不清 → 做成有清晰接口的薄垫片，便宜到随时能拆。



CH2. 从 ReAct 到 TeamAct: 如何让动态能力画像 (Capability Profile) 跑起来

2.1 单个 AI Agent 有一个已被广泛接受的执行模型: ReAct

```
while has_tool_call:
```

Thought → Action → Observation

ReAct 的引擎不是这三步的顺序——是 **Observation** 反向喂 **Thought** 这条反馈回路。单一 Agent 判断任务完成的方式通常是：没有 tool call 了，就生成一段总结。但是这个判断经常是幻觉。在多 Agent 场景下，这个问题被放大了：

- 每个 Agent 都觉得自己做完了，但团队层面没人问“我们的集体目标达到了吗？”
- Agent 之间互相传递状态可以永远循环下去——没有人定义如何让“团队停下来”

2.2 TeamAct 解决了上述问题：团队怎么走完一个完整的循环并停下来。核心是定义了：

- Multi-Agent 之间的信息状态共享机制
- Agent 的状态交接机制

- 团队统一的终止条件

loop:

State → 读共享状态（仓库 / spec / 任务 / 记忆 / 交接胶囊）

Owner → 谁持球？（路由指令 / 显式持有声明）

Action → 持球者执行（写代码 / review / 设计 / 调研）

Evidence → 产出证据（commit / 测试 / trace / 截图）

Verdict → 验证（跨 Agent review / 自检 / CVO 确认）

Route → 传球（路由给下一个 Agent / 继续持有 / 升级给 CVO）

关键定义：Multi-Agent 的六步循环

2.3 Multi-Agent 之间的信息状态共享机制-交接胶囊 (resume capsule)

前一个 Agent 在传球时主动留下一份结构化的摘要——做了什么、为什么、权衡了什么、留下了什么开放问题、下一步该做什么。让接手的 Agent 能快速 bootstrap，这个是协议层的要求。

2.4 Agent 之间的状态交接机制

在大多数多 Agent 系统的 demo 里，Agent 间的任务传递靠的是自然语言约定——在消息里 @ 某个 Agent，对方"看到了"就接手。这在 demo 里 work，在生产中崩溃，因为：

- 消息是非结构化的——无法区分是路由指令还是叙述提及？
- 没有确认机制——对方真的接了吗？还是消息沉了？
- 没有超时——如果对方没响应，谁负责？
- 任务归属不明——"你看一下"之后球归谁？

我们踩过所有这些坑。最终把 Agent 间交接建模成了一个显式的状态机：

- 路由指令必须出现在行首，不能嵌在句子中间（句中的 @ 是叙述，不是路由）
- 任务归属声明是第一人称的——只能声明"我接这个"，不能声明"这是你的"

- 每次交接进入统一的调度队列，不是消息流里的文本约定
- 三种合法的响应：**接**（我做X）/ **退**（这个应该给Y）/ **升级**（需要产品负责人拍板）

这暴露了四种 Multi-Agent 独有的失败模式——这些在单 Agent 系统中不存在：

失败模式	表现	解决方案
球掉地上	没有 Agent 接任务，消息沉了	超时检测 + 自动升级
乒乓球	两个 Agent 互相传但都不干活	实质产出检测（见下文）
虚空传球	路由给一个不存在/不在线的 Agent	路由前校验可达性
角色错配	把前端任务路由给只擅长后端的 Agent	能力声明 + 路由建议（见下文）

表：四种 Multi-Agent 独有的失败模式

2.4.1 乒乓球问题

乒乓球（两个 Agent 互相传球但都不产出实质工作）是我们遇到的最隐蔽的失败模式。对这类问题的修正，不能简单地给一个截断阈值，因为正经的 review 也需要连续传球——reviewer 提 P1，author 修复，reviewer 确认，author 追加测试——这些来回每一次都有实质产出。

我们的办法是：看**每次传球是否伴随实质工具调用和有内容的输出**。真正的乒乓球 signature 是"短文本 + 零工具调用"——这是 RLHF 训练出的"接一句"惯性反射。正经 review 链每一轮都有 commit、测试运行、文件读写。

这背后有一条设计原则：**好的 harness 给 Agent 数据（"你这轮的 tool_call 数为 0，输出只有 3 行"），不给 Agent 结论（"你在乒乓球"）**。结论让 Agent 自己从数据里得出。用正则或小模型替 Agent 判断意图——这种做法看起来省事，但它把判断权从 Agent 手里拿走了，也就拿走了 Agent 自我纠正的机会。

2.4.2 角色错配问题

最初的解决方案是硬编码的——用正则扫描消息内容，看到 "coding"/"merge" 之类的关键词就硬拦某些 Agent。这在两个 Agent 时勉强 work。第三个 Agent 加入后就崩了——新 Agent 的能力边界跟硬编码的关键词表不匹配。于是角色门禁从"硬编码正则"退役，替换为**数据驱动的能力声明**：每个 Agent 的能力限制作为配置数据注入，让 Agent 在收到任务时自己判断"这个我能做吗"。零代码改动就能扩展到任意数量的 Agent。

这与乒乓球问题讲的是同一件事：**harness 不应该替模型思考，而应该让模型在正确的坐标系里思考**。给数据、给约束、给反馈——但把判断留给模型。

2.4.3 双向辩论协议：Generator 有 Push Back 的权利

Anthropic 五种模式中的 Generator-Verifier 有一个隐含假设：Verifier 是权威的。Generator 产出，Verifier 判定，Generator 照做。这里存在一个结构缺陷：**Verifier 也会犯错**。如果 Generator 只能接受不能反驳，错误的 verdict 就没有纠错机制（特别是跨厂商场景下，Verifier 自己的训练盲点可能恰好命中这个 case）。

所以我们将 push back 写进了协作协议的底层：任何 Agent 在任何角色下都有权 push back——前提是带着**证据 + 适用性论证 + 替代方案**。这不是客气的建议，是协议层的硬要求。没有证据的 push back 不合法；有证据的 push back 必须被正视。

2.5 终止条件

五项终止条件（缺一不可，这是 TeamAct 的核心新增）：

- **验收标准全部达成**——不能有 "deferred" 的验收条件
- **证据已附**——每条验收标准都有 commit / 测试 / trace 作为锚点
- **跨 Agent 交叉验证**——非作者的 Agent 确认（不能自己 review 自己）
- **无悬空任务归属**——所有 open question 都已 resolved 或已升级
- **愿景收敛**——首席愿景官的确认不能被 proxy 替代（"CI 通过了"≠"愿景方向对了"）

2.6 一条消息驱动的 TeamAct：一个真实的案例

场景：首席愿景官（CVO, Chief Vision Officer）在微信群发了一句话——“F207 的设计稿我看了，可以开工。”

State：消息如何变成 Agent 的上下文。这句话不是直接被 Agent 看到的。它先经 IM connector 接收，转成结构化消息写入统一的调度队列。队列支持 source、category、priority 三个维度——不是按消息来源排出固定阶梯，而是按语义紧急度分级：CI 失败、PR 冲突、review 变更等触发 urgent；常规对话和定时任务走 normal。CVO 的方向信号在愿景语义上权重最高，但这是路由策略的判断，不是队列实现的硬编码层级。

被调度到的 Agent 启动后，第一步不是读这条消息——是读 shared state。它拉取 git 仓库最新状态（feature spec、任务列表、积压清单），查询记忆系统里与 F207 相关的上下文（之前的讨论、设计决策、开放问题），读取前一轮交接时留下的结构化胶囊（如果有的话）。所有 Agent 看到的是同一份 shared state，不是各自对话历史里的版本。这是 TeamAct 和“群聊协作”的本质区别——shared state 是 grounding 源，不是消息流本身。

Owner：谁持球？看到消息的 Agent 不一定是应该持球的 Agent。“可以开工”是起始信号，但谁来开工需要显式声明。日常的接球方式是行首 @ 路由——前一个 Agent 在消息行首写 @ 目标句柄，目标 Agent 被调度后选择接球（开始行动）、退球（这个应该给别人）或升级（需要 CVO 拍板）。文本描述“我先看看”不构成接球——只有实际的行动或显式的退回/升级才算完成了球权转移。

还有一种场景：Agent 需要退出当前会话等待外部条件（CI 完成、CVO 确认、定时唤醒）。这时用结构化的持球注册工具声明等待原因、下一步计划和预期唤醒时间——相当于分布式系统里的 lease + 定时唤醒，不让球在等待期间无声掉落。

两种机制解决两类问题：行首 @ 解决“球传给谁”，持球注册解决“等待期间球在哪”。早期我们只有前者，频繁出现虚空持球——Agent 说“我来做”然后退出了会话，其他 Agent 都认为有人在做，但没有任何注册信号证明任务还在有人管。

另一个对称的失败是乒乓球与 @ 自身的死锁。在消息行首写 @ 是路由指令——把球传给对方。但如果同一条消息里既 @ 了对方又说“我自己接着做”，两个信号矛盾：@ 说“给你”，正文说“归我”。对方收到后不知道该接还是不接，退球回来又触发新一轮互传。解决办法是把它写成协议层的互斥约束：@ 和“我持球”在同一条消息里不能共存。

Action → Evidence：执行产出证据。持球者开始工作——写代码、跑测试、提 commit。每一步操作都留在 git 和 trace 里。其他 Agent 不需要“被通知”进度——下一次被调度时，从 shared state 读到的就是最新的。

Verdict: 跨 Agent 验证。 代码写完后需要发起跨 Agent review。路由指令出现在消息行首——写目标 reviewer 的具体 Agent 句柄，不是角色名。行首是硬要求——嵌在句子中间的 @ 只是叙述提及，不触发路由。Reviewer 拉取 shared state、读 diff、产出结论：approve 或 blocking。没有第三种。"approve 附带后续建议"是被明确禁止的中间态——reviewer 是守门员，不是 cheerleader。

Route: 传球给谁不是猜的。 Review 通过后，球怎么传？三个合法选项：继续持有（我还有事要做）、传给指定 Agent（行首 @）、升级给 CVO。不合法的是"传给大家看看"——那等于把球扔地上。

真正复杂的场景发生在跨线程。假设 F207 做到一半，持球者发现 F188 有一个接口没闭环，阻塞了当前工作。跨线程通知通过结构化消息工具完成——指定目标线程和目标 Agent，消息到达后系统自动注入回复线索（来源线程、发送者身份），让对方回复时能路由回来。**但跨线程消息只是通知。**真正的阻塞依赖必须同时写进 feature 文档或任务状态——消息是即时的、易丢的；文档是持久的、可追溯的。两边都写，才不会出现"口头说了依赖但没人记得"的状态丢失。



CH3. Harness: 让模型完成现实闭环

3.1 模型本身是一个不具备改变现实的智能体，它能推理，能生成方案，能解释自己为什么这么做；但它天然没有稳定的现实感知、现实记忆、现实行动后果、现实验证，也没有可靠的人机边界。

这是 **Harness** 要解决的问题：让模型知道外面现在发生了什么，能安全地改变外部世界，并把世界做成模型可以感知、可以行动、可以验证、可以恢复、可以学习的样子。

3.2 模型自身无法解决的五类现实问题

- **状态持久性**。长任务跨 session、跨 thread、跨天推进，模型自己的上下文窗口撑不住。压缩会丢信息，新 invocation 会失忆，同一个 Agent 在不同 thread 里也可能混淆上下文。
- **目标一致性**。Agent 容易漂移、自嗨，或者提前宣布完成。
- **行动可验证性**。Agent 说"我做了"不等于做对了。代码有没有改到正确位置？测试是否真的跑过？review 是否由非作者完成？这些不能靠模型自评。
- **熵增抑制**。长期运行的系统会积累冗余规则、过期文档、临时补丁、重复记忆和坏模式。如果 Harness 只会加规则、不会退役规则，迟早会变成另一种技术债。
- **人机边界**。什么时候 Agent 可以自主推进，什么时候必须停下来交给给人？模型没有稳定的"我现在不该继续"自觉。尤其是不可逆操作、愿景级决策、跨 Agent 僵局，必须有外部边界。

3.2.1 感知现实：Durable State Surfaces

第一层是让 **Agent** 不再失忆上岗。我们没有把"状态"只放在对话历史里。对话历史是最脆的状态表面：会被压缩，会被截断，会因为换 carrier 或新 session 丢失。真正的共享现实放在更持久的地方：

- feature spec 记录目标、验收标准、开放问题和完成证据
- git 记录实际修改、提交历史和可回滚边界
- task / queue 记录谁在等、谁在跑、谁被阻塞
- thread / session trace 记录 Agent 做过什么、调过什么工具、留下什么回执
- memory federation 记录决策、教训、方法论和跨域知识
- handoff capsule 记录交接时的 What / Why / Tradeoff / Open / Next

多 Agent 协作不是靠群聊记忆维持的，而是靠共享现实维持的。Agent 每次启动，不是只看最近一句话，而是重新接到这套持久状态表面。这里的 tradeoff 很清楚：状态越外部化，系统越可恢复；但状态表面越多，也越需要治理。

3.2.2 改变现实：Tool Mediation

第二层是行动。Agent 真正进入现实，不是因为它会说话，而是因为它能调用工具。工具越多，Harness 越重要：能力半径变大后，边界必须显式化。

这一层的权衡是：工具层越强，越不能让工具靠模型自觉管理。危险操作需要沙箱、权限、回调 token、幂等约束和人工确认。否则工具越多，副作用越不可控。

3.2.3 验证现实：Evidence & Sensors

第三层是验证。现实闭环里最重要的一句话是：做了不等于做对了。我们需要有证据链：

- 代码修改要有 commit
- bug 修复要有先红后绿的测试
- 合入前要过 quality gate
- 自己写的代码不能自己 review
- 跨 Agent review 要给出 approve 或 blocking，不允许"approve 但后续再说"
- 关键任务需要截图、日志、trace 或 outputVerified 这类外部信号

不同 carrier 的 trace 覆盖度不同。有的能捕获每次工具调用的完整输入输出，有的只能记录 invocation 级摘要。这个差异不能被抹平，所以文章里也必须诚实：目标不是录下一切，而是让关键行为可追溯。

3.2.4 约束现实：Governance Boundary

第四层是治理边界。最典型的问题是上下文压缩。长任务跑久了，runtime 会压缩早期内容。Agent 后半段突然违规，很多时候不是它故意违规，而是规则从它的世界里消失了（被压缩了）。

本文的做法是把关键治理沉到压缩免疫层：身份、家规、任务交接规则、不可逆操作边界、质量纪律进入 native system role / developer role，而不是每轮塞在 user message

里。表层提示会变薄，但底层治理要变厚。同时，这不是把所有东西都塞进 system role。我们做过一次双向手术：

- 删掉与团队文化冲突的主观行为指导
- 保留客观能力指令
- 注入团队自己的治理协议

这里的 tradeoff 是自主性和边界。太软，Agent 会漂；太硬，Agent 会失去判断力。好的治理不是替模型思考，而是让模型在正确的坐标系里思考。

3.2.5 人机边界：Runtime 逃生舱

第五层是人类的入口。人类不是在系统外面等结果，而是在关键边界进入模型闭环。

有些必须交给人类确认的边界可以提前写进规则：删除数据、force push、改生产 Redis、合第三方 PR、砍掉整块功能。还有些边界只能在 runtime 发现：Agent 正在错误坐标系里越跑越远，或者正在把"未做"包装成"已规划"。

这就是 Magic Words 的位置。这是人到 Agent 的 runtime 协议：

- "第一性原理"：停下来，检查是不是在用复杂度代偿无知
- "我能猜出来"：停下来，读真相源，别用推理替代查询
- "下次一定"：停下来，能做的现在做，不能做的走明确 signoff
- "星星罐子"：P0 不可逆风险，立刻停止新增副作用

这背后是一种严肃的设计：人类必须能用极低带宽打断 Agent 的错误轨迹。关键约束也必须写清楚：这些逃生舱只在首席愿景官当前指令中触发。引用、复述、讨论历史中的短语不触发。否则治理协议本身会变成误触源。

3.2.6 清理现实：Entropy Control

长期系统真正难的不是加能力，而是持续清理自己。Harness 有两种折旧：第一种是模型变强后吞掉表层脚手架；第二种更隐蔽：新 Agent 加入后，暴露出某些"通用规则"其实只是对旧 Agent 坏习惯的个体补偿。

所以，Harness 需要熵控机制：

- hotfix 合入后两周自动触发升级 review：正式修复、接受永久方案、已不再相关，三选一
- Build to Delete 类型的规则要标 sunset，不允许无限期占用注意力预算
- root md 和 system prompt 要定期瘦身，避免重复注入和副本漂移
- 记忆系统要治理权威性、触发方式和生命周期，过期知识不能永远排在前面
- 新 Agent 加入后，要重新评估哪些规则是跨 Agent 资产，哪些只是个体补偿

多 Agent 在这里不只是能力升级，还是治理信号源。不同 Agent 的坏直觉不同：有的会糊弄式最小修复，有的会推迟闭环，有的会在错误坐标系里加 fallback，有的会创意漂移。同一条规则在不同 Agent 身上的命中率不同，这正好帮助我们区分公共资产和个体补偿。

治理规则	糊弄惯性	推迟闭环	错误坐标系补丁	创意漂移	治理归类
"脚手架"	高	低	中	低	类型补偿：偏糊弄惯性型
"第一性原理"	中	高	高	高	跨 Agent 资产
"follow-up 尾巴" 禁令	低	高	中	低	类型补偿：偏推迟闭环型
"我能猜出来 / 碎片够了"	中	高	低	低	类型补偿：偏读源不足型
SOP 导航表	低	中	高	高	双重身份：新手路标 / 老手冗余

表：Agent 问题与治理矩阵

3.2.7 适配现实：Harnessability

最后一层是 Harnessability：不是每个系统都同样适合交给 Agent。更适合被 harness 的系统通常有几个特征：

- 有稳定 API 或工具接口
- 有事件流或回调，不需要 Agent 一直忙等

- 有持久状态，可以从中恢复上下文
- 有可验证输出，例如测试、日志、PR、审计记录
- 操作尽量幂等，失败后可以重试或补偿
- 权限边界清楚，高风险动作能被拦截或人工确认

这解释了我们为什么不断做 connector、event bus、trace、feature doc、quality gate 和 memory federation。不是为了给系统堆复杂度，而是在把外部世界改造成 Agent 可感知、可行动、可验证、可治理的现实表面。

到这里，Harness 的位置就清楚了：它不是模型能力的补丁，而是模型与现实之间的运行时。表面的提示词会变薄，底层的现实闭环会变厚。模型越强，越需要这层运行时来承载它改变现实的半径。

CH4. Memory: 多域记忆运行时

4.1 为什么 RAG 会输给 grep

Grep 赢在三件事：

- **当前性**。它搜的是当前文件系统，不怕向量索引没更新
- **精确性**。字面命中就是命中，不会自信地召回一段"语义很像但其实不对"的文本
- **可审计性**。命中在哪个文件哪一行，Agent 可以立刻打开原文，不需要相信黑盒排序。

很多 RAG 在项目知识场景里输给 grep，不是因为向量检索没价值，而是因为它丢了项目知识最重要的工程属性：文件路径、行号、标题层级、更新时间、文档类型、决策权威、上下文关系。一个被切碎的 chunk 可能语义相似，但它未必是最新真相源，也未必是该被相信的那份文档。

但是 grep 也有天花板：

- **需要先验关键词**。不知道"桌宠"在代码里实际叫 voice companion 或 desktop shell，就很难搜到。
- **没有语义桥接**。搜"注意力缺陷"未必能命中写成 AUDHD 的经验记录。
- **不知道权威性**。旧草稿、废弃 spec、已批准 ADR，在 grep 结果里都只是文本。
- **不知道跨域关系**。项目仓库、个人上下文、金融知识库、虚拟世界设定、GitHub/Gmail connector 里的信息不会自然联邦。

- **没有结果反馈。**某条知识被搜到后有没有被读、有没有真的帮助任务成功，grep 完全不知道。

我们的路线是：保留 grep 的真相源感和可审计性，再补上语义检索、治理、跨域联邦和运行时反馈。

对于决策、教训、spec 这类**知识文档**，真相源仍然是仓库里的 markdown 文件——Agent 能读、能改、能用 git 追溯。数据库和向量索引只是编译层，坏了可以从 markdown 重建。这是一个有意的设计决策：**知识的真相源不能是只有系统能读的格式**。人类产品负责人要能直接打开文件看到团队记忆里写了什么。

但记忆系统里还有另一类数据：Agent 的检索行为和任务轨迹（搜了什么、读了什么、结果如何）。这些**运行时 trace** 不是 markdown 编译出来的，而是系统在 Agent 执行过程中自动采集的 runtime evidence，存在独立数据表里。两类数据的真相源不同，治理方式也不同。

4.2 记忆系统的三个入口

记忆系统有三个检索入口，对应三种不同的认知需求：

- **精确导航** (graph_resolve)：你知道要找什么——一个 feature 编号、一个决策文档、一个已知的锚点。系统返回这个锚点以及它的关联关系。这解决的是"我知道大概在哪，帮我展开上下文"。
- **零先验扫描** (list_recent)：你不知道要找什么——可能是刚从上下文压缩中恢复，可能是接手了一个不熟悉的任务。系统按时间倒序列出最近被索引的文档 (feature / decision / lesson / plan)、讨论摘要 (session digest) 和记忆条目——注意这是**已索引文档的最近视图**，不是原始聊天消息或记忆存储的全量扫描。这解决的是"发生了什么？我错过了什么？"
- **语义搜索** (search_evidence)：你知道要找什么方向但不知道确切的锚点——用关键词或自然语言描述查询。系统用关键词匹配 (BM25) 和语义向量的混合模式检索，结果带治理元数据：置信度、权威等级、文档类型。这解决的是"关于 X 我们知道什么？"

4.3 从 grep 到记忆系统的七步演进

记忆系统不是一次设计出来的。它是沿着"grep 够用在哪里、不够用在哪里"一路长出来的。内部 feature 编号只是证据锚点，真正的演进线是下面七步：

F102: 检索底座。 先把知识从"散落在文件里、靠人 grep"升级成可重建的索引：markdown 仍是真相源，SQLite / 全文检索 / 向量检索 / RRF 融合排序（把多个排序列表按名次合并）是编译产物。这一步解决的是"能不能搜到"。

F163: 治理层。 搜到不等于可信。我们给知识加三类元数据：**权威性**（authority，谁能证明它成立，比如铁律、已验证决策、候选观察）、**触发方式**（activation，什么时候应该出现，比如永远在场、按任务范围、只在查询时出现）、**生命周期**（status，它现在是有效、待复核、已失效还是归档）。这一步解决的是"搜到了该不该信"。

F169: 记忆反射愿景。 能搜不等于会在正确时机想起来。F169 提出的是：记忆不该只是书架，而应该像 Agent 的外部工作记忆反射。这里引入了 **salience**（任务相关性）：同一条高权威知识，在不同任务里相关性不同；相关的应该浮上来，不相关的应该暂时降下去。这一步解决的是"什么时候该想起什么"。

F152: 外部项目冷启动。 记忆第一次走出本项目。Agent 去到一个陌生仓库时，通用扫描器读取 README、代码结构、变更记录和散落文档，带 provenance（来源溯源）进入索引；在外部项目踩到的经验再回流到全局方法论层。这一步解决的是"新项目怎么不从零开始"。

F186: 多域知识联邦。 知识边界从"一个项目仓库"扩展到多个知识域。我们引入 **Collection（知识域）**：一个有独立真相源、负责人、扫描器、敏感级别和治理策略的知识域。项目、全局方法论、个人上下文、虚拟世界设定、金融/医疗/生物实验设计这类专业资料库，都可以注册成知识域。检索时按权限和任务需要跨知识域并发查询，再融合排序。这一步解决的是"知识不在一个地方怎么办"。

F188: 管护与佩戴协议。 图书馆建好了不等于 Agent 会用。F188 正在把三个检索入口、知识图谱、健康检查、搜索技能和图形导航做成 Agent 面向的使用界面。这里的"佩戴协议"不是把 top-k 硬塞进上下文，而是让不同模型知道什么时候该精确查、什么时候该扫最近、什么时候该语义搜索，搜完还要追源读原文。这一步解决的是"能力有了，Agent 会不会用"。

F200: 反馈闭环。 最后开始闭合循环：用 Agent 的真实行为判断知识价值。搜到了有没有读？读了有没有用？用了之后任务有没有被验证？这些信号反过来调整导航排序。注意，这只影响"下次更容易找到什么"，不改变知识本身的权威性。这一步解决的是"什么知识真的有用"。

一句话总结：**F102** 让知识可搜，**F163** 让知识可信，**F169** 让知识按任务浮现，**F152** 让知识可迁移，**F186** 让知识可跨域，**F188** 让 Agent 会用，**F200** 让系统知道什么真的有用。

4.4 最终设计：多域记忆运行时

这七步长出来的最终形态，不是一个 RAG 工具，而是一个六层的多域记忆运行时：

- **真相源 / Collection 层**：项目 spec、决策、教训、讨论、个人上下文、外部仓库、虚拟世界、专业领域知识库、GitHub/Gmail/IM 等外部连接器。每个知识域有独立负责人、敏感级别和治理规则
- **扫描 / 编译层**：markdown、仓库结构、connector 数据和运行时 trace 被扫描进 SQLite / 全文索引 / 向量索引 / 图谱索引。索引可重建，真相源不藏在黑盒数据库里
- **联邦检索层**：scope 管检索表面 (docs / threads / memory)，dimension 和 collections 管知识域路由；跨知识域并发检索 + RRF 融合；有权限和敏感级别过滤
- **治理层**：权威性（谁能证明）、触发方式（什么时候出现）、生命周期（是否仍有效）、来源溯源、过期检测、矛盾检测、回滚。这是和普通 RAG 最大的差别之一
- **Agent 佩戴协议层**：三个检索入口 + 搜索技能 + 任务相关性提示，让不同能力的 Agent 都能有效使用记忆，而不是被动吃 top-k
- **反馈闭环层**：行为指标、消费加权排序、TaskTrajectory（任务轨迹）、outputVerified（任务结果验证信号）。回答"这条知识是否真的改变了行动结果"

4.5 检索驱动的适配循环

记忆系统的反馈闭环构成了一种区别于梯度更新训练循环的学习范式：**检索驱动的适配循环**。区别是根本性的：

- 训练循环改变模型的潜在能力和泛化边界；检索循环改变模型能接触到的知识库
- 训练循环要等完整的训练运行才能生效；检索循环即时生效——一个 Agent 提炼的教训，下一个 Agent 立刻搜得到
- 训练循环和特定模型绑定；检索循环跨厂商通用——同一套教训对不同模型都有用
- 训练循环有灾难性遗忘的风险；检索循环没有——知识是文本，加新的不会覆盖旧的
- 训练循环的改动难以审计和回滚；检索循环完全可审计——教训是人类可读的 markdown，可以阅读、修改、删除

它可以让多个不同厂商的 Agent 共享同一套项目知识、教训和决策上下文——这是训练循环做不到的。每次任务的检索链条（搜了什么→读了什么→改了什么→结果如何）被持久化记录，这些 trajectory 本身就是有价值的适配信号。

4.6 简单系统 + 聪明 Agent

记忆系统有一条贯穿始终的设计原则：**系统保持简单，把智能留给 Agent**。记忆系统只负责 "给了关键词，返回匹配结果"——不试图猜 Agent 想找什么。

这个原则有一个实践验证：团队里不同 Agent 用同一个搜索工具时，会基于各自的领域理解做不同的查询扩展，搜出不同的子集。如果系统层面做了查询改写或意图分类，就会把所有 Agent 的搜索行为同质化——反而损失了多 Agent 的检索多样性。

怎么教 Agent 有效搜索？不是在系统里加逻辑，是写一个搜索技能模块：先展开同义词、再分 doc/thread 分别搜、搜完对比覆盖度、不足的角度再补一轮。这个技能和 Agent 的模型能力结合——聪明的 Agent 搜得更好不是因为系统给了更好的结果，是因为它问了更好的问题。

4.7 总结：我们的记忆系统

数据源是多域联邦——项目内文档 + 讨论 + trace（Agent 自己产出的）为核心域，同时联邦式链接个人上下文、外部世界设定、专业领域知识库。有权威等级 + 生命周期治理 + 消费加权排序。解决"多 Agent 长程协作怎么不漂、怎么复用、怎么跨域导航"。

两个独有特性：

第一，**知识生产者 = 知识消费者**。Agent 写教训 → Agent 搜教训 → 绕开坑。其他系统的核心闭环不是"生产者自己在任务结果中反向改写排序"——RAG 和 Agentic Search 的知识来自外部；LLM Wiki 可以编译项目内资料，但编译者和日常任务执行者通常是分离的。我们的知识来自团队自身的实践，生产和消费在同一循环里——这意味着知识的质量和团队的实践质量直接挂钩。

第二，**有运行时结果反馈闭环**。搜了、用了、结果好 → 排名上升。RAG 和 Agentic Search 是读多写少的系统。LLM Wiki 有知识维护循环（lint 查矛盾和过期），但缺"任务结果反向决定知识排序"的运行时反馈。我们的差异不在检索算法，在于**真相源、权威、溯源、使用反馈、结果验证**都在同一个闭环里。

CH5. Eval: Harness 的自我代谢系统

5.1 核心思想

有 harness，就必须有 eval。否则 harness 只会增生，不会代谢。

5.2 benchmark 的局限性

Benchmark 只能测试模型的能力，没有办法测试 harness 与模型的契合度。第 1 章的公式说了：最终效能 = 能力 × Harness 契合度，**Benchmark** 只测了等号左边的一个因子。本文对这个问题的答案是一组运行时控制回路。

5.3 三层 eval

层	对应建设	它回答什么	现在谁在消费
观测底座	F153: 运行时可观测基础设施	发生了什么？工具调用、耗时、失败、重试、健康状态	所有 eval 的传感器层
Harness / A2A (Agent 间协作) Eval	F192: 社会技术 harness eval	协作协议、任务交接、等待外部条件时的任务持有机制、harness 组件有没有改善 workflow	harness-fit-digest 定时后台任务和 harness 评审
Memory Eval	F200: 记忆召回 eval	搜索结果有没有被读、有没有帮助完成任务、记忆排序是否真的有用	memory recall 指标和排序策略

表：三层 eval

F153 是描述性观测层：告诉我们发生了什么，不判断好坏。F192 和 F200 才是在这些观测数据之上做解释。

F192 的试点对象是 A2A（Agent 间协作）链路。它先在 spec 里写下预期声明：这个 harness 组件应该服务谁、什么时候触发、摩擦指标是什么、回归用例是什么、什么情况下应该退役。然后从 F153 拉运行时数据，检查实际行为和预期是否一致。此外，不可观测本身就是一种 eval 结果。不知道一个组件是否正常，和组件已经故障一样需要行动。

F200 的试点对象是记忆系统。它不让模型自评"这条记忆有用吗"，而是看 Agent 真实行为：搜到了有没有读，读完有没有继续追溯，是否又改 query，是否退回 grep。这些行为信号不会替代权威性判断——一条铁律不能因为没人搜就降权——但它能告诉我们：导航层是不是把有用的东西放到了 Agent 面前。

5.3.1 Eval Contract：每块 harness 的接口

F192 里最重要的发明不是某个指标，而是 **Eval Contract**。这个词可以翻译成 "预期声明"：你创建新的 harness 时，必须同时写清楚它想改善什么。一份合格的预期声明至少回答五个问题：

问题	人话解释
服务谁	哪类 Agent / 哪类任务会用到它
何时触发	什么时候它应该发挥作用
摩擦指标	它失败时会留下什么信号
回归用例	怎么证明它没有退化
退役信号	什么情况下它应该被删除或精简

表：一份合格的预期声明

最后一项最关键，把第 1 章的 Build to Delete / Built to Persist 判别落到了操作层：

- 如果一块机制的触发率持续下降，且同类任务没有变差，它可能该退役。
- 如果一块机制长期高频触发，且摩擦率下降，它可能值得加固。
- 如果一块机制从来不触发，比"指标下降"更危险——它可能是死代码，只是在占用 Agent 的注意力预算。

5.3.2 三方信号：只看 trace 也不够

运行时数据能告诉你很多事，但不能告诉你全部。我们采用三类信号交叉：

- **第一方：首席愿景官的愿景信号。** 一个 feature 做完了，CVO 打开看——满意吗？和他脑子里的画面差多远？这是无法完全算法化的判断。trace 可以全绿，但如果他说"不是我要的感觉"，那就是没有完成。
- **第二方：Agent 作为 harness 用户的摩擦信号。** Agent 是 harness 的一线用户。工具好不好用、规则清不清晰、环境约束卡不卡，Agent 最有体感。这里不是让 Agent 写自由散文式反思，而是在特定条件下做结构化采访：比如某个工具重试三次，或者某类交接反复失败，就对着 trace 问"你当时卡在哪里？"
- **第三方：运行时观测信号。** 工具调用模式、失败频率、重试次数、耗时分布、读写链路。这些数据不经过主观过滤——记录的是实际发生了什么。

三方信号的交叉才是真 eval。CVO 满意 + Agent 顺手 + trace 异常，说明有隐藏技术债；CVO 不满意 + Agent 顺手 + trace 正常，说明问题可能在愿景翻译或品味层，而不是工具层。

5.3.3 问题归因

Agent 做的不好，可能是工具没接好、规则错位、需求翻译错了，甚至是首席愿景官自己还没想清楚。本文归因与 7 类：

类别	含义	改什么
愿景缺口	愿景方向本身没想清楚	愿景判断
翻译偏差	spec 把愿景翻译偏了	需求文档
harness 错位	机制设计在错误坐标系上	工具 / 规则重新定位
工具缺口	要做的事没有工具支撑	新增或改造工具
执行缺口	工具可用但 Agent 没用对	技能文档 / 搜索策略

类别	含义	改什么
环境漂移	同样操作在新环境里变了	环境约束 / connector
品味落差	功能对，感觉不对	审美标准 / CVO 反馈

表：7 类归因矩阵

每条归因输出结构化记录：证据来自哪条运行时数据，Agent 或 CVO 反馈了什么，归到哪一类，建议行动是什么，状态有没有推进。结构化不是为了好看，是为了之后能统计：哪些问题是工具缺口，哪些是愿景翻译偏差，哪些发现提出来以后没人执行。

5.3.4 动态 eval 轨迹

Eval 的产物不只是一个结论。更有价值的是轨迹：Agent 的意图是什么、选了哪个工具、读了哪些文件、在哪个分支失败、怎么恢复、最终谁验证了结果。

	静态 benchmark	eval 轨迹
给的信息	输入 / 输出	意图 / 工具选择 / 失败分支 / 读了什么 / 改了什么 / 谁验证 / 怎么恢复
反映	通过率	真实工作全过程

表：与 benchmark 记录作对比

一类研究和产业实践已经开始从静态 benchmark 转向轨迹采集：搭仿真环境，让 Agent 在里面跑任务，采集完整轨迹。方向是对的，但成本结构不一样。

维度	仿真采集路径	工程现场自然产出
数据来源	专门建仿真 app	真实协作的副产品
采集成本	高：先建仿真环境	边际成本低：协作天然产 trace

维度	仿真采集路径	工程现场自然产出
真实性	仿真用户行为，分布会飘移	in-the-wild revealed preference
数据归属	取决于研究方和平台协议	用户本地拥有、可选择导出
跨厂商	通常单一仿真平台	多厂商 Agent 在同一工作流里留下轨迹

表：产业实践与工程产出轨迹对比

但 raw trace 也不够。一堆无结构日志和一条有价值的 eval 轨迹之间，差了一层**类型化加工**：搜索链路、文件读写、结果验证、归因类别。

5.4 终态：Harness Eval Control Plane（harness 评估控制面）

现在的 F192、F200 还是两条并行的垂直竖线：各自有指标、各自有定时任务、各自消费自己的数据。这是启动期合理形态，但不是最终架构。终态应该是一个横切的 **Harness Eval Control Plane（harness 评估控制面）**。它不是单纯的指标看板，也不是给人看的漂亮指标页，而是 harness 生命周期的控制面：哪块机制正在增值，哪块机制正在折旧，哪条发现需要行动，哪个模型或运行载体（例如不同 CLI / provider 接入方式）在某类任务上成为瓶颈。

CH6. Multi-Agent 的可靠性建设

6.1 Multi-Agent 本质是一个分布式系统

Multi-Agent 会遇到：多个独立的执行上下文，共享的可变状态（代码库、配置、文档、记忆库），异步的通信通道（消息传递、@ 路由、工具调用）这其实是一个**分布式系统**。基于**能力画像 (Capability Profile)** 驱动的团队，比 **Role Agent** 团队更依赖状态连续性。

行业大多数 multi-Agent 讨论把可靠性挑战压在 prompt 和 orchestration 层面处理。但分布式系统的核心教训是：**你不可能消除故障，你只能设计对故障的容忍。**

6.2 三类可靠性挑战

6.2.1 单 Agent 长任务的持久性

生产中遇到三种故障模式：

- 任务已执行但通道断了（文件已写，不能盲目重试）
- 本地执行器报告成功但远程报告失败（异步状态机 race condition）；
- provider 返回空响应（需要理解错误语义，不是简单重试）。

这三种指向一个结论：**controller 与 Agent 的通道需要解耦，不能决定任务的生死，任务状态需要独立的持久化**。本文借鉴分布式系统，给出了如下的解法和恢复分级原则：

组件	解决什么	借鉴的分布式系统原则
持久化监管者	任务状态与通道解耦，不过期	借鉴 Saga 协调器：把长事务拆成可补偿步骤
副作用日志	每个外部操作被记录，恢复时知道"哪些已做"	类似数据库预写日志（Write-Ahead Log）的副作用记录
安全恢复	幂等（重复执行结果不变）+ 去重，按风险分四级	默认拒绝未知情况，逼近"恰好执行一次"
结构化恢复卡	中断时给用户结构化状态，不是自由文本错误	补偿/恢复计划视图

表：借鉴分布式系统的可靠性解法

等级	操作类型	恢复策略
Tier 1	读取 / 构建 / 测试 / lint	始终自动恢复
Tier 2	自有沙箱 / worktree / 可确定性探测	探测成功后自动恢复

等级	操作类型	恢复策略
Tier 3	共享文件 / 外部服务 / GitHub 写操作	不自动恢复，出恢复卡
Tier 4	force-push / merge / release / 高风险不可逆操作	永远不自动恢复，dispatch 前硬拒

表：恢复的分级原则

- 分级原则是 fail-closed，也就是**默认拒绝**：遇到未知操作类型默认归入最高限制，不是最低。

6.2.2 跨 Agent 协作一致性

回到 Ch.2 的 TeamAct 主循环——State → Owner → Action → Evidence → Verdict → Route。Route 阶段一只 Agent 把任务交接给另一只，这个切换瞬间是**最脆弱的窗口**。前一只做完 Action 但没来得及更新 Evidence 就崩了，后一只接球看到不完整状态；后一只正在启动但 liveness 误判它"已失联"，触发错误重分配。

这是分布式系统里的经典问题：**在没有全局事务的系统里，怎么保证状态转移的原子性？**数据库领域有成熟方案（两阶段提交、Raft 共识），但 Agent 协作的特殊性在于：参与者是不同厂商的 LLM，你控制不了它们的内部行为，只能通过 **shared state** 的协议来约束。

一般地说：多 Agent 的共享状态（代码库、任务列表、讨论记录、记忆库）就是分布式系统的 shared mutable state。Ch.2 的 TeamAct 协议本质上是一个轻量协作状态机——通过 Evidence 和 Verdict 的显式记录让多 Agent 对当前状态有一致理解。它不提供 Raft 级别的强一致保证，但在 Agent 协作的容错需求下够用。当状态机被打断，就需要恢复机制来重建一致性。

这里要补一条边界：**弱状态机不是放养**。Cat Café 的协作状态机故意保留判断力，因为开放式任务的路径不能提前画死；但一旦进入严肃流程 (*同时具备几类特征：有外部副作用，失败不容易撤销，需要权限和审计*)，执行权就要交给确定性的 workflow。Agent 负责理解目标、判断是否该进入流程；skill 负责告诉 Agent 这个流程需要哪些前置条件、检查点和退出条件；workflow 负责真正可靠地执行。

6.2.3 跨 provider 的语义一致性

当团队里有 Claude Agent、GPT Agent 和 Gemini Agent 时，每个 provider 的超时策略、错误码语义、通道协议、恢复机制都不一样。一只 Claude Agent 崩了，接手的 GPT Agent 能不能从同一个副作用日志恢复？日志本身是 provider 无关的——但“超时了怎么办”“崩了怎么恢复”“怎么判断是否存活”这些运维语义在不同 provider 上不一致。

这就是为什么需要统一宿主抽象：不同 provider 共享同一套控制面（传输 × 绑定 × 运行时契约 × 事件适配器），监管者作为独立伴生进程（sidecar）运行，不和任何具体 provider 耦合。不是让所有 provider 变一样——是让运维决策有统一的实现。

6.2.4 不可控与可控：三类挑战贯穿始终的区分

不可控的：provider 上游的稳定性、网络质量、上游的超时策略。这些是环境约束。行业里很多可靠性讨论都在抱怨 provider 不稳定——“Claude 又 500 了”“GPT 的 stream 又断了”。抱怨是对的，但抱怨不是架构。

可控的：你这层的 liveness 判断、状态持久化、副作用追踪、恢复策略、协作协议。这些全在你的代码里。

架构是假设不可控的一定会出问题，然后设计可控层的容错能力。分布式系统几十年的实践就是在做这件事。multi-Agent 面对的是同一类问题——节点从服务器变成了 LLM Agent，通道从 TCP 变成了 stream/SSE，共享状态从数据库表变成了代码库和知识库。工具不同，原则一样。

6.3 可靠性解锁任务深度

可靠性基础设施反过来影响 Agent 的表现：当 Agent 知道副作用被追踪、恢复是安全的、中断不等于失败，它可以执行更深的任务——不需要人为把任务拆碎来规避超时风险。可靠性没有改变模型的能力，但它改变了 Agent 能持续发挥能力的时间窗口

CH7. 基于能力画像(Capability Profile)构建的 Multi-Agent 能力涌现

7.1 关键表现

能力涌现发生在输出分布被系统改写之后：

- **右尾变长**：不同 Agent 提供不同路径，最好的候选解更容易出现
- **左尾被截断**：错误要连续穿过 review、测试、证据、恢复、CVO 拉闸才会抵达用户
- **方差被吸收**：某个 provider 或某只 Agent 的波动变成系统内部的返工成本，而不是用户可见的质量崩塌

7.2 上限：团队不是平均值，而是候选路径的最大值

单 Agent 做复杂任务，本质上是在一条路径上推进。多 Agent 不是把三份答案平均，而是让多个不同认知路径同时出现，然后用 review、证据和愿景判断筛选修正。可以用一个粗糙但有用的公式建立直觉：

$$\text{上限收益} \approx \max(\text{不同 Agent 提出的候选路径})$$

这个 max 成立的前提不是 Agent 数量多，而是路径足够不同。一个团队真正要问的是：这个 Agent 会不会带来新的视角？

7.3 下限：错误要连续穿过多层门，才会抵达用户

团队系统里，一个错误要变成用户可见错误，需要连续穿过多层门：author 犯错、reviewer 没抓住、测试 / trace 没暴露、shared state 没留下证据、eval 没归因、CVO 没拉闸。这不是严格独立事件，所以不能把它当精确概率公式。但它表达了一个工程事实：只要这些门的盲点不完全相关，最终错误率就不会等于单个 Agent 的错误率。我们简化公式：

$$\text{用户可见错误} \approx \text{author 错误率} \times \text{reviewer 漏检率} \times \text{盲点相关性}$$

这里的“盲点相关性”不是实验测出来的常数，而是一个工程提醒：跨厂商、跨角色、跨工作习惯的价值，在于把这个系数压低。同质化团队的问题不是 Agent 少，而是相关性高。

7.4 波动吸收：模型质量变成内部成本

有 harness 的世界里，波动会先变成内部成本：

- 模型忘了上下文，记忆联邦把事实找回来
- Agent 写偏了，review 退回
- 任务中断了，可靠性控制面留下恢复点

- 某个工具或规则不再有效，eval 触发 sunset review
- 某个 provider 当前不适合这个任务，调度换路径

所以用户感知的质量不再等于某个模型的即时状态，而更像：

用户感知质量 = 模型能力 × harness 契合度 × 纠错链路 × 恢复能力

这就是第 1 章公式在收束章里的升级。我们补充了反面定义：

好的 Environment Fit 不只在模型变强时放大收益，也在模型波动时降低传导

7.5 Token 账单

一个常见挑战是：三只 Agent 协作，token 消耗是三倍；我只用一只最强 Agent，是不是更便宜？这个挑战的陷阱在于，它只算了**显性 token**，没有算**复杂任务的完整成本**。

真正的成本为：

总成本 = token + 返工成本 + 人类心智负载 +

跑偏后发现太晚的尾部成本 + 错误进入真实环境后的修复成本

单 Agent 常常在前半段显得便宜：少一轮 review，少一次讨论，少一个候选路径。但如果它在第 80% 的地方才跑偏，人类才发现方向错了，返工成本可能远高于前面省下的 token。

Multi-Agent 协作的价值不在于每一步都更便宜，而在于把方向错误、事实错误、架构错误更早暴露出来。早暴露的错误便宜，晚暴露的错误昂贵。

所以判断标准不是“三只 Agent 是否比一只 Agent token 少”，而是：**多出来的协作 token，是否买到了更早的纠偏、更少的人类路由、更低的返工尾部风险？**

人类有价值的点不在于**消息总线**，而是**方向锚点**。人类消息转述保留的信息，不如 Agent 直接读取同一份 shared state。而在**长链路协作中**，**保真率是连乘项**。当然，Multi-Agent 不是默认正确。四种结构会亏：

- **盲传**：后一棒不是纠错，而是在没有新信息的情况下重做
- **伪拆分**：任务拆了，但每个子任务没有变简单，只多了协调税

- **同质化**：所有 Agent 盲点高度相关，review 只是重复同一类判断
- **协调税超过收益**：正确率只提高一点，但 token、等待和人类心智负载翻倍所以协作要被 eval，而不是被信仰。赚就保留，不赚就收缩。

7.6 双层语言：内部高密度，外部讲人话

用户可能会观察到另一个现象：Agent 之间协作时会用很多内部缩写、Feature 编号、review 严重度、commit、AC、trace。对人类读者来说，这像黑话；对 Agent 系统来说，它是高密度协议。

这不是问题本身，问题在于有没有翻译层。

一个成熟系统应该有两种语言：

- **Agent-to-Agent 语言**：高密度、精确、可追溯，适合协作
- **Agent-to-human 语言**：少黑话、讲因果、给判断，适合决策

如果强迫 Agent 之间全程讲人话，协作会变慢、信息会变粗；如果把内部语言原样丢给人类，用户会看不懂。Cat Café 的做法是允许内部高密度协作，但在交付、人类决策、正式文章里强制翻译。人类不需要理解每个内部缩写，但必须能理解系统最后为什么这么判断。

7.7 最小必要复杂度

TeamAct 解决任务交接；harness 解决模型如何接入现实；记忆联邦解决事实和经验如何复用；eval 解决哪些机制该改、该删、该加固；可靠性控制面解决长任务中断和跨 provider 语义差异，每一层都在解决一个特定的问题。

但最小必要复杂度不是静态的。模型会变强，某些脚手架会过时；团队会扩张，某些协议会变得必要；任务会跨域，某些记忆域会被接入。复杂度必须被 eval 持续审查，而不是一旦建成就永久保留。**这也是全文的最终答案：**

团队质量 = 上限搜索 × 下限保护 × 状态保真 × 失败恢复

总成本 = token + 返工 + 人类心智负载 + 尾部风险